

PROXMOX AUTOMATION SERIES · PHASE
2 ADDENDUM

Storage, Hardened Verify + Grow

Three additions on top of the field-verified storage stack: one group_vars list as the single source of truth, a playbook that reboots the VM to prove fstab and LVM boot persistence, and an opt-in fifth disk that exercises the growth workflow end to end.

PVE 9.2 · TERRAFORM 1.15 · BPG/PROXMOX 0.111.1

ANSIBLE-CORE 2.20 · REBOOT MODULE · LVS JSON

AUGUST 2026 · COMPANION REPO: PROXMOX-AUTOMATION

BOOT PERSISTENCE · SINGLE SOURCE · FIFTH
DISK

Contents

1. What This Addendum Adds	1
2. One Source of Truth for the Storage Layout	3
3. Proving Boot Persistence: 03_storage_verify.yml	5
4. Growing Beyond Four Disks: The scratch01 Workflow	9
5. Field Runbook and Transcripts	12
6. Troubleshooting the New Pieces	15

1. What This Addendum Adds

This addendum is the third module of the Proxmox Automation Series. The Phase 2 guide ended with the storage stack verified in the field: four serial-tagged disks converged into mounted XFS filesystems, a resize demonstrated online from 20G to 30G, and real transcripts behind every claim. That is a solid place to stand, and this document builds on it rather than replacing it. Three additions close the gaps that remained open when the Phase 2 guide went to print.

Two of those gaps are claims the stack makes but never tested. First, the mounts are supposed to survive a reboot: `fstab` lines exist, LVM is supposed to activate its volume groups at boot, and until somebody actually reboots the machine and checks, that is belief rather than evidence. Second, the growth story covers growing an existing disk, while production growth often arrives as a NEW disk: a fresh LUN for a new workload. The third gap is structural: the storage layout lives as variables inside the build playbook, which was fine for one playbook and becomes a copy-paste hazard the moment a second playbook needs the same list.

The addendum closes all three in the same spirit as the rest of the series: small, reviewable moves that keep the working parts working. The disk layout moves to `group_vars/provisioned.yml` as the single source of truth. A new playbook, `03_storage_verify.yml`, reboots the VM and proves boot persistence with markers, facts and assertions instead of hope. And an optional fifth disk, enabled by a `tfvars` knob, exercises the attach-to-mount path end to end. The diff inventory is deliberately small; the field-verified core of `02_storage.yml` is untouched, and the additions have since been field-run too: the verify playbook earned one line of correction (chapter 3), and the growth workflow completed without needing its fallback stop/start (chapter 4).

File	Change	Purpose
ansible/inventory/group_vars/provisioned.yml	new	The storage layout as one list: serial, vg, lv, mount. Read by the build and verify playbooks
ansible/playbooks/02_storage.yml	modified (v3)	The data_disks list moved out to group_vars; a guard assert fails with a clear message if the file is missing. All tasks unchanged from field-verified v2
ansible/playbooks/03_storage_verify.yml	new (1.1)	Boot-persistence verification: markers, reboot, fresh facts, assertions, receipts. Edition 1.1 adds follow: true to the stat task, the one-line fix the first field run earned
terraform/variables.tf	modified	One new knob: scratch_disk_size_gb (default 0 = no fifth disk)
terraform/vm.tf	modified	A dynamic disk block creates scsi5 with serial scratch01 only when the knob is greater than zero
terraform/terraform.tfvars.example	modified	The new knob documented with the full growth workflow around it
docs/phase-2-addendum-guide.pdf	new	This document

Table 1. The complete addendum diff: three new files, four modified.

Scope and boundaries

As always in this series, the additions are scoped so that each one can be verified alone. The verification playbook does one reboot run and checks one machine; it is not a monitoring system, and it does not schedule itself. The growth workflow adds exactly one disk and reuses the existing build playbook unchanged. The boundaries below are the honest ones; everything outside them stays where the Phase 2 guide left it.

In scope for the addendum	Deliberately out of scope
One group_vars list shared by build and verify playbooks	Dynamic inventory from Terraform state (Phase 4)
Boot persistence proven by a scripted reboot run	Continuous monitoring of mounts (Zabbix arrives in Phase 3)
An optional fifth disk via a tfvars knob	Disk removal or shrink (still impossible by design)
Field transcripts for 02, 03 and the growth run (ok=15 changed=1; ok=28 changed=2 failed=0; growth: 02 ok=15 changed=5)	The no-reboot mode transcript (verify_reboot=false: documented, not yet field-logged)

Table 2. What this addendum builds, and what it leaves alone.



Figure 1. The addendum in one picture: one list feeds build and verify; the reboot must give everything back; the fifth disk exercises growth.

2. One Source of Truth for the Storage Layout

The Phase 2 playbook declared its disk list in a `vars:` block at the top of the play. That is idiomatic Ansible and perfectly readable, and for a single playbook it is the right shape. The problem appears the moment a second playbook wants the same information: the verification playbook needs the same four serials, the same VG and LV names and the same mount points, because proving the stack means checking exactly what the build playbook claims to have built. Two copies of one list in two files is how drift is born: add a fifth disk, update one playbook, forget the other, and the verification quietly stops covering it.

The fix is the standard Ansible answer: group variables. The inventory already defines a group called `provisioned`, so a file at `inventory/group_vars/provisioned.yml` is loaded automatically for every host in that group, before any playbook code runs. Both playbooks now simply use `data_disks` and the guarantee is structural: there is exactly one list in the repository, and it is the one both playbooks read.

ANSIBLE/INVENTORY/GROUP_VARS/PROVISIONED.YML (THE WHOLE IDEA)

```

---
# Single source of truth for the Phase 2 storage layout.
# Read by playbooks/02_storage.yml (build) and
# playbooks/03_storage_verify.yml (prove it survives a reboot).

data_disks:
- serial: var01 # scsi1, size: var_disk_size_gb
  vg: vg_var
  lv: lv_var
  mount: /srv/var
- serial: log01 # scsi2, size: log_disk_size_gb
  vg: vg_log
  lv: lv_log
  mount: /srv/log
- serial: data01 # scsi3, size: data_disk_size_gb
  vg: vg_data
  lv: lv_data
  mount: /srv/data
- serial: backup01 # scsi4, size: backup_disk_size_gb
  vg: vg_backup
  lv: lv_backup
  mount: /srv/backup

# 03_storage_verify.yml reboots the VM by default. Quick check
# without the downtime:
# ansible-playbook playbooks/03_storage_verify.yml -e verify_reboot=false
verify_reboot: true

```

Two details in that file carry design weight. The comments keep the scsi interface and the tfvars knob next to each serial, because the file is the contract between the Terraform side and the Ansible side and reviewers should be able to check both halves in one place. And `verify_reboot` lives here rather than buried in the playbook so the one behavior-altering knob of the verification run is visible exactly where an operator goes looking for settings.

Moving the list out of the playbook changes what happens when the file is missing, so both playbooks open with a guard. A run from the wrong directory, or against an inventory that lacks the group, would otherwise die mid-play with an undefined variable at whatever task touches the list first; the guard fails first, with a message that names the file and the likely cause.

THE GUARD BOTH PLAYBOOKS OPEN WITH

```

- name: Require the storage layout (group_vars/provisioned.yml)
  ansible.builtin.assert:
    that: data_disks is defined and data_disks | length > 0
    fail_msg: >-
      data_disks is not defined. The storage layout lives in
      ansible/inventory/group_vars/provisioned.yml; check that the file
      exists and that the play is running against the `provisioned`
      group with the repository's ansible.cfg.

```

The precedence ladder is worth knowing before experimenting. Play-level `vars:` would override `group_vars`, which is exactly why the list was removed from the play rather than kept as a default; the file has no competition left. Extra vars beat everything, which makes them the right tool for one-off experiments without touching the source of truth.

Precedence (low to high)	What it means here
<code>group_vars/provisioned.yml</code>	The single source of truth; both playbooks read it as-is
play vars: in the playbook	Would OVERRIDE <code>group_vars</code> ; removed from <code>02_storage.yml</code> precisely so it cannot
<code>--extra-vars</code> on the command line	Beats everything; the documented escape hatch for experiments (<code>verify_reboot=false</code>)

Table 3. The relevant slice of Ansible variable precedence for this design.

Two playbooks, one list

The build side and the verify side read the same file. A disk added to `group_vars` is built by 02 and verified by 03 on their next runs; neither playbook can quietly fall behind the other.

3. Proving Boot Persistence: 03_storage_verify.yml

The Phase 2 field run proved the stack in the state it was left in: mounted, sized, and readable through `df`. It did not prove the stack comes back after a reboot, and that is a different claim in kind, not just in degree. `fstab` entries are only read at boot. LVM activation at boot is a separate mechanism from the explicit activation the build triggered. A mount that works because a playbook ran `mount` by hand is indistinguishable from a persistent one right up until the first reboot takes the service down for good. In production DB-server practice, "the machine rebooted and the data disks did not come back" is a classic 3 a.m. page, and it is almost always a one-line mistake: a typo in `fstab`, a VG outside the activation scope, a filesystem type the boot sequence will not mount.

The verification playbook does what a careful engineer would do by hand, but repeatably and with receipts. It mints a one-run token and writes a marker file into every data filesystem **BEFORE** the reboot, so persistence can be checked by content rather than by directory names; a freshly reformatted filesystem would have the same mount point and different bytes. It snapshots the serial to device mapping from the kernel view. It reboots the VM through the `reboot` module, which handles the shutdown, the wait for

SSH to return and a post-boot command test. Then it proves, in order: markers byte for byte, every mount mounted from refreshed facts, every logical volume active, every fstab line present, and every serial resolved again.

MARKERS BEFORE THE REBOOT

```
- name: Mint a one-run verification token
ansible.builtin.set_fact:
  verify_token: "phase2-{{ ansible_facts.date_time.epoch }}-{{ 100000 | random }}"

- name: Write a marker file into every data filesystem
ansible.builtin.copy:
  content: "{{ item.mount }} {{ verify_token }}"
  dest: "{{ item.mount }}/phase2-verify.txt"
  mode: "0644"
  loop: "{{ data_disks }}"
```

The token exists so each run verifies its own write. Comparing against a fixed string would pass even if the file were stale from a previous run; comparing against a token minted seconds before the reboot means the bytes read after the boot were written before it, by this run, to this filesystem. The marker content is the mount path plus the token, so a marker that somehow ended up on the wrong filesystem fails the comparison as well.

THE REBOOT, AND ONE SUBTLE FACT ABOUT FACTS

```
- name: Reboot the VM (the moment of truth for fstab and LVM)
ansible.builtin.reboot:
  msg: "Phase 2 verification reboot: proving fstab + LVM boot persistence"
  pre_reboot_delay: 5
  post_reboot_delay: 10
  reboot_timeout: 600
  test_command: "uptime"
  when: verify_reboot | default(true) | bool

- name: Refresh facts after the reboot (pre-reboot facts are stale)
ansible.builtin.setup:
  when: verify_reboot | default(true) | bool
```

The facts-refresh task looks redundant and is not. `gather_facts` runs once at play start, so `ansible_facts.mounts` still describes the pre-reboot machine afterwards; asserting against it would prove nothing about the boot. Re-running `setup` after the reboot makes the mount check read the world as it is now. The `| bool` on the guard is deliberate too: extra vars arrive as strings, and in Jinja the string "false" is truthy, so the naive check would reboot precisely when asked not to.

ASSERTIONS AFTER THE REBOOT (MARKERS SHOWN; MOUNTS AND LVS FOLLOW)

```

- name: Read the marker files back
ansible.builtin.slurp:
src: "{{ item.mount }}/phase2-verify.txt"
loop: "{{ data_disks }}"
register: marker_read

- name: Marker contents must match byte for byte (data persisted)
ansible.builtin.assert:
that: marker_read.results[idx].content | b64decode
== item.mount ~ ' ' ~ verify_token
loop: "{{ data_disks }}"
loop_control: { index_var: idx }

- name: Every data filesystem must be mounted after the reboot
ansible.builtin.assert:
that:
- ansible_facts.mounts
| selectattr('mount', 'equalto', item.mount) | list | length == 1
- (ansible_facts.mounts
| selectattr('mount', 'equalto', item.mount)
| map(attribute='fstype') | list) == ['xfs']
loop: "{{ data_disks }}"

```

The active-volume check made one honest design decision worth recording. LVM reports volume state inside the `lv_attr` attribute string, where the fifth character encodes the active state; parsing attribute positions is exactly the kind of cleverness that breaks silently when a tool changes its report layout. The playbook instead reads the `lvs` JSON report to count the expected volumes, and asks the filesystem directly: an active logical volume exposes its device node as `/dev/vg/lv`, an inactive one has no node at all, so a `stat` per volume is both the simpler and the more honest test.

The first field run then taught this check one more lesson, and it is worth recording because no syntax check can see it. The run was green through the reboot: markers matched byte for byte, all four mounts were up from fresh facts, the LVM report held exactly the expected volumes. Then the active-volume assert failed on all four volumes at once with the machine demonstrably healthy, because `/dev/vg/lv` is a udev-managed symlink to `/dev/dm-X` and the `stat` module does not follow symlinks unless told to: without `follow` it stats the link itself and reports `islnk`, never `isblk`. The failure was in the check, not in the stack, and the surrounding green assertions proved that on their own: XFS cannot mount through an inactive volume. Edition 1.1 adds the one parameter, the second field run closed green at `ok=28 changed=2 failed=0`, and the troubleshooting table in chapter 6 keeps the row so the pattern stays recognizable for anyone writing their own `stat`-based checks.

COUNTING LVS FROM THE JSON REPORT, THEN TESTING ACTIVITY BY DEVICE NODE

```
- name: Read the LVM logical volume report (JSON)
ansible.builtin.command:
cmd: lvs --reportformat json --noheadings -o vg_name,lv_name,lv_attr
register: lvs_state
changed_when: false

- name: Parse the LVM report
ansible.builtin.set_fact:
lv_rows: "{{ (lvs_state.stdout | from_json).report[0].lv }}"

- name: Exactly the expected logical volumes must exist
ansible.builtin.assert:
that: lv_rows | length == data_disks | length

- name: Check every logical volume's device node
ansible.builtin.stat:
path: "/dev/{{ item.vg }}/{{ item.lv }}"
follow: true # /dev/vg/lv is a symlink to /dev/dm-X
loop: "{{ data_disks }}"
register: lv_nodes

- name: Every logical volume must be active after the reboot
ansible.builtin.assert:
that:
- lv_nodes.results[idx].stat.exists
- lv_nodes.results[idx].stat.isblk
loop: "{{ data_disks }}"
loop_control: { index_var: idx }
```

The fstab receipt greps for the exact line the mount module wrote: the device path exactly as given (`/dev/vg_data/lv_data`, not the `/dev/mapper` alias) followed by the mount point. Matching both device and mount point catches the two real failure shapes at once: the line missing entirely, and the line present but pointing somewhere else. The receipts at the end of the run print `findmnt` per mount, the `lsblk` serial view, and `df`, so the log of a green run doubles as documentation of the machine state.

Every check degrades gracefully when the reboot is skipped. Running with `-e verify_reboot=false` turns the run into a no-downtime smoke test: mounts, LVs, fstab lines and serial resolution are all still checked against the live machine, and only the marker comparison loses its meaning, because the write and the read happen in the same session without a boot in between. The table below states exactly what each check proves in both modes; the distinction matters and the playbook does not pretend otherwise.

Check	With the reboot (default)	With <code>verify_reboot=false</code>
Marker files byte for byte	Data persisted across the boot	Same-session smoke test only
Mounts mounted, fstype xfs	fstab boot-time activation worked	Current mounted state is correct
LV device nodes exist	Boot-time LVM activation worked	LVs are active right now
fstab lines present	Persistence is declared, not just live	Same
Serials resolve to one disk	Identity scheme survived the boot	Identity scheme holds now

Table 4. What each assertion proves with and without the reboot.

Should survive, verified to survive

fstab and LVM activation only prove themselves at boot. The verify playbook spends one reboot to turn an assumption into a receipt, and after any storage change, re-running it is the cheapest insurance in the pipeline.

4. Growing Beyond Four Disks: The scratch01 Workflow

The Phase 2 resize demo answered the most common storage request: the same disk, bigger. The second most common request looks different: a NEW disk for a new workload, a fresh LUN from the storage team with its own mount point. In the manual world that means a console dance on the hypervisor and a second dance in the guest, and it is precisely where the device-letter guessing game from chapter 2 of the Phase 2 guide used to bite hardest, because a new disk shifts the enumeration of every disk after it. The growth workflow in this addendum exercises that path declaratively: one tfvars knob enables the disk, one apply attaches it, one list entry and one playbook run mount it.

The Terraform side is a conditional block. HCL has no "create this block only if" switch on a literal block, so the canonical pattern is a `dynamic` block whose `for_each` is a one-element list when the knob is set and empty when it is not. With the default of zero, the plan contains no `scsi5` disk at all; the knob does not create a stub or a placeholder, it creates nothing.

TERRAFORM/VM.TF (THE OPT-IN FIFTH DISK)

```

variable "scratch_disk_size_gb" {
  description = "Optional fifth data disk (serial: scratch01), GB. 0 = off."
  type = number
  default = 0
}

# in resource "proxmox_virtual_environment_vm" "test_vm":
dynamic "disk" {
  for_each = var.scratch_disk_size_gb > 0 ? [var.scratch_disk_size_gb] : []
  content {
    datastore_id = var.storage_id
    interface = "scsi5"
    size = disk.value
    serial = "scratch01"
    ssd = true
    discard = "on"
  }
}

```

The Ansible side needs no new code at all, only a new row in the contract. Terraform owns which disks exist and how big they are; the `group_vars` list owns what each disk becomes inside the guest; the serial is the join key between the two halves. That split is what keeps the growth workflow small: the build playbook already loops over whatever list it finds, so a fifth entry is three lines of YAML, and the verify playbook covers the new disk on its next run without anyone telling it to.

Side	Owns	Where it lives
Terraform	Which disks exist, interfaces, sizes, serials	terraform/vm.tf + tfvars knobs
Ansible	What each serial becomes: vg, lv, mount point	inventory/group_vars/provisioned.yml
The kernel	Which device letter each serial is right now	Resolved fresh every run, never persisted

Table 5. The two-sided contract; the serial is the join key.

One field finding from the first Phase 2 run now becomes an explicit caveat instead of a surprise. Resizing an existing disk while the VM runs is fully online, as the 20G to 30G demo proved with the machine up. A newly ATTACHED disk is different: PVE writes the new disk into the VM configuration, but the serial reaches the guest kernel reliably only at the next full VM start, because the running QEMU process predates the configuration change. A guest-OS reboot does not help; QEMU survives it. So the honest expectation after enabling `scratch01` on a running VM is: the disk may appear in `lsblk` immediately, and if its serial does not, the fix is one stop and start on the node, not debugging. The growth field run then landed on the sunny side of that may: the apply attached `scsi5` to the running VM, the serial was visible to the kernel immediately, and the whole attach-to-mount path completed without the stop/start. The fallback stays in the table and the troubleshooting chapter anyway, because one sunny run narrows a caveat; it does not retire it.

Operation	VM state	Serial visible in guest	Action if not
Resize existing disk (data 20 to 30)	Running throughout	Immediately (kernel view refreshes)	None needed; field-proven online
Attach new disk (scsi5 scratch01)	Running	May require full VM start (field run 1: immediate)	On the PVE node: qm shutdown 200 && qm start 200, then re-run 02
Anything after a full VM start	Started fresh	Always (config and QEMU agree)	Diagnose: qm config 200 grep serial vs lsblk

Table 6. Resize versus attach: the asymmetry the field taught, and the growth run refined.

THE GROWTH WORKFLOW, END TO END

```
# 1. on ctrl-01: enable the disk (terraform/terraform.tfvars)
scratch_disk_size_gb = 5

# 2. apply; the disk attaches while the VM runs
cd ~/proxmox-automation/terraform && terraform apply

# 3. append the layout entry (ansible/inventory/group_vars/provisioned.yml)
# - serial: scratch01
# vg: vg_scratch
# lv: lv_scratch
# mount: /srv/scratch

# 4. build the mount (same playbook, one more loop iteration)
cd ../ansible && ansible-playbook playbooks/02_storage.yml

# 5. if step 4 reports scratch01 NOT FOUND: on the PVE node
# qm shutdown 200 && qm start 200 # full VM start, not a reboot
# then re-run step 4

# 6. prove it, including across a reboot
ansible-playbook playbooks/03_storage_verify.yml
df -h /srv/scratch
```

Resize online, attach (usually) online too

Growing an existing disk stays fully online. Attaching a NEW disk was online in the growth field run as well; the one documented fallback, a full VM start, exists for the day the serial lags behind the config.

The apply that attaches the fifth disk carried one more field lesson worth a paragraph, because the first sight of it raises an eyebrow: the plan announces 1 to destroy. The resource being destroyed is not the

VM; it is the downloaded cloud image. Upstream had re-published the Ubuntu cloud image between runs, the file size changed from 861837824 to 864818688 bytes, and `proxmox_download_file` treats a size mismatch as a reason to fetch it again. The apply therefore spent five minutes re-downloading an artifact before spending seconds attaching the disk; the VM itself was updated in-place, no destroy touched it, and the warning printed right above the plan says exactly this and names the switch that pins the artifact (`overwrite=false`) for anyone who prefers reproducibility over freshness. Reading the plan before approving it, the habit this series has pushed since Phase 1, is what turns that eyebrow into a shrug.

5. Field Runbook and Transcripts

Series convention, two editions later: every transcript in this chapter is real, including the growth run. The build and verify runs happened on `ctrl-01` against `vm-test-01`, and the verify run needed two attempts for the best possible reason: the first attempt caught a bug in the check itself, the stat and follow lesson from chapter 3, the one-line fix landed on the control node, and the second attempt ran green end to end. The growth run then attached a fifth disk, mounted it and proved it across a reboot without needing a single fallback step, and its verification reboot rotated every device letter on the way, which turns out to be the best possible final exam for the identity scheme. The only mode without a field log left is the no-reboot smoke test, documented in Table 4 and awaiting a casual run. Nothing in this chapter is expected shape anymore.

Machine	Command	Paste back	Status
ctrl-01	<code>cd ~/proxmox-automation && unzip -o proxmox-automation.zip</code>	unzip tail (files replaced)	done
ctrl-01	<code>cd ansible && ansible-playbook playbooks/02_storage.yml</code>	PLAY RECAP	field: ok=15 changed=1
ctrl-01	<code>ansible-playbook playbooks/03_storage_verify.yml</code>	mapping debug lines, assert lines, PLAY RECAP	field: ok=28 changed=2 failed=0
ctrl-01	<code>ansible-playbook playbooks/03_storage_verify.yml -e verify_reboot=false</code>	PLAY RECAP of the no-reboot mode	optional, not yet run
ctrl-01	<code>edit tfvars: scratch_disk_size_gb = 5, then terraform apply</code>	apply output (plan: image refresh + scsi5, VM in-place)	field: done (5 min: image re-download)
ctrl-01	<code>add scratch01 to group_vars, re-run 02_storage.yml</code>	mapping line for scratch01, PLAY RECAP	field: ok=15 changed=5

Table 7. The addendum field runbook, complete: every required step has its field log.

The verify transcript below is the first green run, on the four-disk stack, trimmed to the lines that carry information: the mapping as it stood before the reboot, the reboot itself, the assertion that cost the first attempt its green, and the before-and-after mapping that closed the run. Assertion tasks are quiet when green, so the interesting lines are the debug messages and the recap; the full run prints every assert per disk, and the trim keeps one of each shape. A failed assertion stops the run at the exact layer that failed, with a message naming the machine-side check for that layer.

FIELD TRANSCRIPT: THE FIRST GREEN VERIFY RUN (FOUR DISKS, TRIMMED)

```
TASK [Show the mapping before the reboot] *****
ok: [vm-test-01] => (item=var01) => {
  "msg": "var01 = /dev/sde"
}
... (log01 = /dev/sdd, data01 = /dev/sdc, backup01 = /dev/sdb)

TASK [Reboot the VM (the moment of truth ...)] *****
changed: [vm-test-01]

TASK [Every logical volume must be active after the reboot] ***
ok: [vm-test-01] => (item=vg_var/lv_var) => {
  "msg": "All assertions passed"
}
... (vg_log/lv_log, vg_data/lv_data, vg_backup/lv_backup)

TASK [Show the mapping before vs after ...] *****
ok: [vm-test-01] => (item=var01) => {
  "msg": "var01: /dev/sde before -> /dev/sde after (vg_var/lv_var on /srv/var)"
}
ok: [vm-test-01] => (item=backup01) => {
  "msg": "backup01: /dev/sdb before -> /dev/sdb after (vg_backup/lv_backup on /srv/backup)"
}
... (log01, data01)

PLAY RECAP *****
vm-test-01 : ok=28 changed=2 unreachable=0 failed=0
```

Two numbers in that recap deserve a paragraph each. First, changed=2 is the honest minimum for a verify run: the marker files are written fresh every run, and the reboot is real work the module reports as a change; everything else only asserts or reads, and a third run would land on the same two. Second, ok=28 against ok=18 in the aborted first attempt is not ten extra checks but the same checks finally allowed to finish: the first attempt stopped at the active-volume assert, and the tasks after it, serial resolution after the boot, the before-and-after mapping, the fstab receipts and the closing findmnt, lsblk and df receipts, only ran once the assert passed. The before-and-after mapping also came out identical this boot, letters and all, which is luck rather than a guarantee; the identity scheme does not need stable letters, only stable serials, and either outcome is a green run.

The growth run then supplied the other half of that evidence. With five disks attached, the verification reboot rotated every device letter: var01 moved from sde to sdf, each of its neighbors shifted one seat,

and scratch01 came back as sdb. The run stayed exactly as green, because nothing in the stack addresses a letter: fstab names /dev/vg/lv devices, LVM finds its volumes by metadata written on the disks themselves, and the playbooks resolve serials fresh from the kernel view on every run. That is the identity thesis of the whole Phase 2 design, demonstrated at the worst case, a reboot that reorders everything, rather than at the friendly one. One operational footnote from the same run: the df receipt belongs to the guest. Running `df -h /srv/scratch` on the control node answers with a confused "No such file or directory", which is correct, the mount does not exist there; the receipt was read over SSH from the machine that owns the mount.

For the growth run, the recap arithmetic deserves a sentence before the transcript. `changed=5` is exactly the five tasks that do first-run work for a new disk: the physical volume task (whose `pvresize` cosmetic change lands on every run anyway), the volume group, the logical volume, the filesystem and the mount. The four existing disks contributed no changes of their own; their VG, LV, XFS and mount items all reported ok, converged and untouched. And the verify recap did not move at all, `ok=28 changed=2`, the same two changes, because a fifth list entry adds loop iterations, not tasks: the playbook grows with the machine at zero new moving parts.

FIELD TRANSCRIPT: THE GROWTH RUN (TRIMMED)

```
TASK [Show the serial to device mapping ...] *****
ok: [vm-test-01] => (item=scratch01) => {
  "msg": "scratch01 (vg_scratch/lv_scratch -> /srv/scratch) = /dev/sdf"
}

TASK [Create volume groups (one per disk)] *****
changed: [vm-test-01] => (item=vg_scratch)
... (vg_var, vg_log, vg_data, vg_backup: ok, converged)

PLAY RECAP (02_storage.yml: fifth disk built) *****
vm-test-01 : ok=15 changed=5 unreachable=0 failed=0

TASK [Show the mapping before vs after ...] *****
ok: [vm-test-01] => (item=var01) => {
  "msg": "var01: /dev/sde before -> /dev/sdf after (vg_var/lv_var on /srv/var)"
}
ok: [vm-test-01] => (item=scratch01) => {
  "msg": "scratch01: /dev/sdf before -> /dev/sdb after (vg_scratch/lv_scratch on /srv/scratch)"
}
... (log01 sdd->sde, data01 sdc->sdd, backup01 sdb->sdc)

PLAY RECAP (03_storage_verify.yml: fifth disk proven) *****
vm-test-01 : ok=28 changed=2 unreachable=0 failed=0

# receipt, read from the guest over SSH:
# /dev/mapper/vg_scratch-lv_scratch 5.0G 130M 4.9G 3% /srv/scratch
```


6. Troubleshooting the New Pieces

The failures worth pre-registering are the ones the design predicts, plus two the addendum's own field runs contributed. Each row below names the layer that fails, the shape of the message, and the machine-side fix; all of them are aimed at turning a red run into a one-branch decision rather than an investigation. The two rows inherited from the Phase 2 guide (stale QEMU serials, by-id absence) still apply unchanged and are not repeated here.

Symptom	Cause	Fix
scratch01 NOT FOUND right after the apply that attached it	A newly attached disk's serial reaches the guest kernel only at the next full VM start; the running QEMU predates the config change	On the PVE node: <code>qm shutdown 200 && qm start 200</code> , then re-run <code>02_storage.yml</code> (a guest reboot is NOT enough)
<code>data_disks</code> is not defined at the very first task	The play ran from outside the repository's <code>ansible.cfg</code> , so <code>group_vars/provisioned.yml</code> never loaded	Run from <code>ansible/</code> with the repo's <code>ansible.cfg</code> , or pass <code>-i inventory/hosts.yml</code> explicitly
Reboot task times out (<code>reboot_timeout: 600</code> exceeded)	The VM is slow to boot, or SSH did not come back; the module waits for both plus the test command	Re-run; if it persists, boot the VM console via the PVE UI and check for a boot-time hang (<code>fsck</code> , failed mount)
Marker mismatch on a mount	The filesystem mounted there is fresh or wrong, not the pre-reboot one; the data did not persist	Check <code>lsblk SERIAL</code> vs <code>fstab</code> for that mount; the device mounted is not the LV that holds the marker
LV device node missing (not active)	Boot-time LVM activation did not cover the VG	In the guest: <code>systemctl status lvm2-activation.service</code> , <code>vgchange -ay</code> ; then investigate why boot activation skipped it
Assert says the volume group was not activated, yet every mount is up and markers matched	<code>stat</code> on <code>/dev/vg/lv</code> without <code>follow</code> : <code>true</code> stats the symlink itself: <code>islnk true</code> , <code>isblk false</code> ; all four active LVs <code>false-fail</code> together	Add <code>follow: true</code> to the <code>stat</code> task (edition 1.1 ships it); first observed in the addendum field run 1, green after the fix
Mount present in <code>fstab</code> but not mounted after reboot	Boot-time mount failed (wrong options, device not ready in time)	In the guest: <code>systemctl status local-fs.target</code> , <code>journalctl -u local-fs.target</code> ; the <code>verify</code> run names the exact mount
The growth apply announces 1 to destroy and takes about five minutes	Upstream re-published the cloud image; <code>proxmox_download_file</code> sees the size change and re-downloads it (the plan names the file resource, not the VM)	Harmless: the VM update is in-place and the destroy never touches it; to pin the artifact, set <code>overwrite=false</code> in <code>template.tf</code>

Table 8. Pre-registered failures for the addendum pieces.

Where Phase 2 stands now

The storage layer is now complete in both directions the series promised: built declaratively from a golden template and four serials, grown online by a `tfvars` edit, and, with this addendum, provable across a reboot from one command. Edition 3 closes the set: the single-source layout, the verify playbook and the growth workflow have all run in the field, every transcript in chapter 5 is real, and the lessons the runs

taught, stat follows symlinks only when asked, a may-caveat that stayed lucky, an upstream image that refreshed itself mid-pipeline, are written where future runs will find them. Phase 2 is closed. The natural next stop is Phase 3: agent roles, starting with Zabbix Agent 2, which will also give the mounts a permanent watcher instead of a point-in-time check, and rsyslog forwarding toward QRadar, which turns the log disk from a landing pad into part of a pipeline.

One reboot, one receipt

After any storage change, resize or attach, the habit is the same: re-run the build playbook, then re-run the verify playbook. The first keeps the state converged; the second proves the machine would still stand up on its own at 3 a.m.